

Job Submission...

Created by Andrew Johnston, last modified on Mar 14, 2024

Running Jobs on Gadi

To run compute tasks such as simulations, weather models, and sequence assemblies on Gadi, users need to submit them as '**jobs**' to '**queues**'. Each queue has different hardware capabilities and limits. The overall procedure takes a few steps, that we will outline here, but here are a few key points before moving on.

- When submitting jobs, users need to specify the queue, duration, and resource needs of their jobs. Gadi uses **PBSPRO** to schedule all submitted jobs and keeps nodes that have different hardware in different queues. You can read about the hardware available in the different queues on the [Gadi Queue Structure](#) page. Users submit jobs to a specific queue to run jobs on the corresponding type of node.
- When wrapping your tasks up inside a job for submission, you must estimate the amount of computational resources the job will use and for what duration you will need your job to run. This is so that the job scheduler knows what resources to reserve for the job.
- **If the job uses more than it requested, it will be terminated immediately.** See the [Gadi Queue Limit](#) page for more details on resource limits and costs on each type of the nodes.
- Once the requested amount of resources becomes available to the system, the job is sent to the scheduled hosting node(s). If the job requests more than a single node, it is first started on the head node where all the commands in the job submission script are executed one by one. Whether the job can utilise the requested amount of resources or not depends on the commands and scripts/binaries called in the submission script.

- ☐ If there are tasks in the job need that access to the internet at any stage, they have to be separately packed into a job on a copyq node as none of the standard compute nodes have external network access outside of Gadi.

On this page

- [Creating a submission script](#)
- [Interactive jobs](#)
- [copyq jobs](#)

Job Submission

Creating a submission script.

To run jobs on Gadi, users need to create a **PBS submission script**. This script can be written in which ever editor you wish, e.g. **Vim**, **Nano**, etc, and contains simple lines that tell PBS how you want to run your job. To the right is an example of a basic PBS script that outlines some key requests that should be in your script.

1. Specifies which shell to use
2. An allocation of **48 CPU** cores
3. **190 GiB of memory** has been requested
4. **200 GiB** of local disk on the compute node
5. Submitted to the **normal queue**
6. Using project **a00**
7. A walltime of **02:00 hours**
8. Storage in **alldata** and **scratch** to come from project **a00**

☐ **First time running a HPC job?**

Practice downloading, compiling, and running a job on Gadi with our [Hello World introductory lesson](#). It's a great way to take your first steps into high performance computing.

This is just an example submission script, there are

9. and [Pages /](#) / [Using Gadi...](#)
 10. The

python3 to execute the script `$PBS_NCPUS`, and redirects the output of this python script to a log file in **g/data**, in project **a00**, with the name of the **jobID**.

For more PBS directives, please see our [comprehensive list here](#).

Different queues have different hardware and capabilities, make sure you have selected the correct queue by checking our [queue structure](#) and queue [limits pages](#).

Once you have saved this script as a **.sh** file, you will be able to submit it using the **'qsub'** command, followed by the file name,

```
$ qsub <jobscript.sh>
```

After your job has been successfully submitted, you will be given a jobID. This will be a string of number ending in **.gadi-pbs**, for example:

```
12345678.gadi-pbs
```

You can then use this jobID to monitor and enquire about the job that is running. There are several ways to monitor your job over its lifespan on Gadi. Please see our [job monitoring guide](#) for ways to obtain information about your job.

When writing a PBS script, users are encouraged to only request resources that they need, and that will allow their tasks to run close to the 'sweet spot'. This is where the job can take advantage of parallelism and achieve a shorter execution time, while utilising at least 80% of the resources requested. Searching for this sweet spot can take time and experimentation, some code will need several iterations before that efficiency is found.

- ☐ On job completion, by default, the contents of the job's standard output/error stream gets copied to a file in the working directory with the name in the format **<jobname>.o<jobid>/<jobname>.e<jobid>**.

For example, when the job 12345678 finishes, there are two files created with the names **job.sh.o12345678** and **job.sh.e12345678** as the record of its **STDOUT** and **STDERR**, respectively, and these two log files are located inside the same folder where the job was submitted. (**STDOUT** is a normal print out, **STDERR** is an error stream that shows if your job ran into any issues.) We recommend users check these two log files before proceeding with the post-processing of any output/result from the corresponding job.

Interactive Jobs

Interactive jobs allow users to run jobs that can be monitored and adjusted at certain points during their lifespan. Users can submit jobs in an interactive shell on the head compute node, which allows them to test and debug code before running the entire job. NCI recommends that users utilise this resource to debug large parallel jobs or install applications that have to be built when GPUs are available.

Instead of writing a PBS script, Interactive jobs are run as a command on the login nodes. This is done with the command **\$ qsub -I** followed by the parameters you wish to run, for example

```
$ qsub -I -qgpuvolta -Pa00 -lwalltime=00:05:00,ncpus=48,ngpus=4,mem=380GB,jobfs=200
qsub: waiting for job 11029947.gadi-pbs to start
qsub: job 11029947.gadi-pbs ready
```

Here we have a command submitting a job on the **gpuvolta** queue, through project **a00**, requesting **05:00 minutes of walltime**.

It asks for **48 CPUs, 4 GPUs, 380 GiB of memory**, and **200 GiB** of local disk space. Once this job begins, it will mount **/g/data/a00** to the job and enter the job's **working directory**.

When you
something

[Pages](#) / / [Using Gadi...](#)

```
[aaa777@gadi-gpu-v100-0079 ~]
```

This means that you are now logged into a compute node, you can see that change in the example below.

```
[aaa777@gadi-login-03 ~]$ qsub -I -lwalltime=00:05:00,ncpus=48,ngpus=4,mem=380GB,jc
qsub: waiting for job 11029947.gadi-pbs to start
qsub: job 11029947.gadi-pbs ready

[aaa777@gadi-gpu-v100-0079 ~]$ module list
No Modulefiles Currently Loaded.
[aaa777@gadi-gpu-v100-0079 ~]$ exit
logout

qsub: job 11029947.gadi-pbs completed
```

This is a very minimalistic example of an interactive job. by default, the shell doesn't have any modules loaded, if you need to load modules repeatedly inside interactive jobs, you can edit your `~/ .bashrc` file to automatically load them. Once you are finished with the job, run the command

```
$ exit
```

to terminate the job.

Copyq jobs

The login nodes are a shared space, at any time you could potentially be sharing the nodes with hundreds of other users while logged in. To make sure that everyone has fair access to these nodes, any job that runs for more than 30 minutes, or exceeded 4 GiB of memory, will be terminated. if you need to transfer a large amount of data. more that the amount allowed in the login nodes, NCI recommends that you submit it in a job within the copyq queue. Jobs that require any internet access during running or long software installations should also be run through this queue.

To do this, you need you need to write a PBS script detailing that you want to use copyq, along with some components to direct the data into new directories. For example:

```
#!/bin/bash

#PBS -l ncpus=1
#PBS -l mem=2GB
#PBS -l jobfs=2GB
#PBS -q copyq
#PBS -P a00
#PBS -l walltime=02:00:00
#PBS -l storage=gdata/a00+massdata/a00
#PBS -l wd

tar -cvf my_archive.tar /g/data/a00/aaa777/work1
mdss -P a00 mkdir -p aaa777/test/
mdss -P a00 put my_archive.tar aaa777/test/work1.tar
mdss -P a00 dmls -ltrh aaa777/test
```

This job requests the copyq queue to be used, while also giving new commands at the bottom of the script.

- create an archive of the data at **/g/data/a00/aaa777/work1**.
- make a directory **aaa777/test/** using the tane file system

- mo
 - migi
- [Pages](#) / / [Using Gadi...](#)

To compile code inside a **copyq job**, it may be necessary to load modules such as intel-compiler and request more **jobfs** to allow enough disk space to host data written to **\$TMPDIR**.

Authors: Yue Sun, Andrew Wellington, Mohsin Ali, Javed Shaikh,Adam Huttner-Koros, Andrew Johnston

[job-submission](#) [running-jobs](#) [pbs](#) [pbs-script](#) [submission-script](#) [interactive-jobs](#) [copyq](#)